

HỌC TĂNG CƯỜNG

1. Giới thiệu về học tăng cường

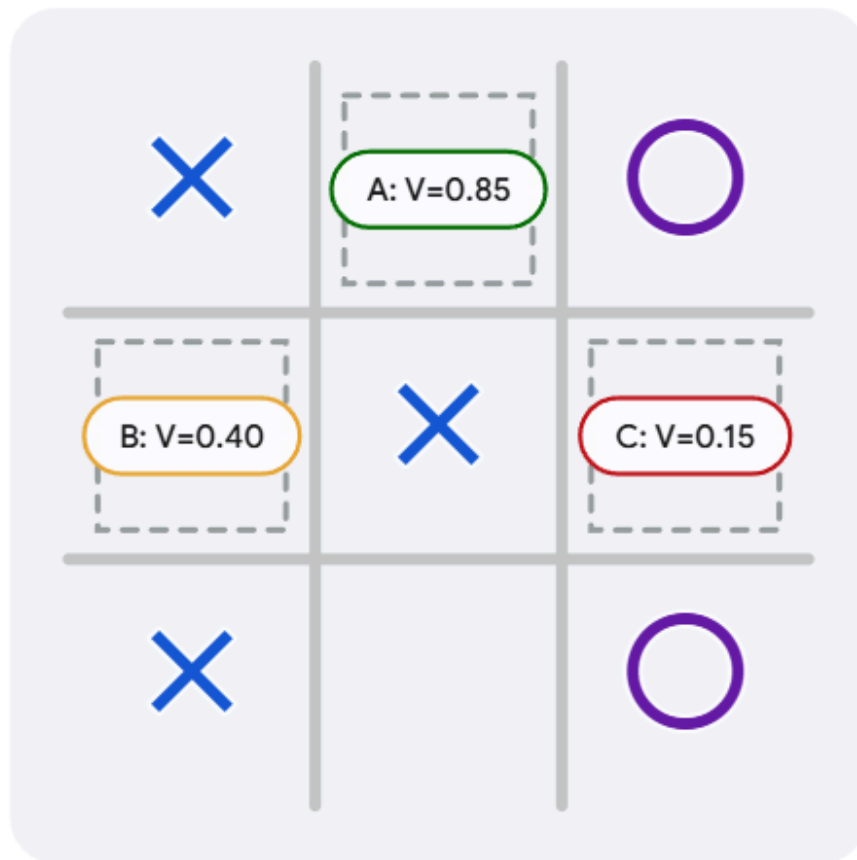
- Học tăng cường là học thông qua việc tương tác với môi trường để đạt được mục tiêu (tối đa hóa điểm thưởng là luôn cố gắng để đạt được nhiều điểm thưởng hơn nhưng không phải là bắt buộc phải đạt được điểm thưởng tối đa vì có nhiều yếu tố cản trở điều này). Giống việc một đứa trẻ tự học mà không có ai hướng dẫn, nó cứ thử sai nhiều lần dần dần sẽ biết được nên làm gì để đạt được mục tiêu, học để biết được trong trường hợp này thì nên thực hiện hành động gì.
- Đặc điểm:
 - Hành động được thực hiện ở hiện tại sẽ ảnh hưởng đến thông tin nhận được trong bước tiếp theo
 - Không có hướng dẫn trực tiếp: tự thử sai nhiều lần và rút ra được kinh nghiệm là thực hiện hành động nào thì sẽ có được nhiều điểm thưởng nhất
 - Hành động đưa ra không chỉ ảnh hưởng đến phần thưởng ngay lập tức mà ảnh hưởng đến toàn bộ điểm thưởng nhận được trong tương lai
- Điểm khác so với các loại học khác:
 - Học có giám sát: học với tập dữ liệu có gán nhãn, còn học tăng cường thì không có hướng dẫn cụ thể nào cả vì không thể thu thập được hết mọi trường hợp có thể xảy ra
 - Học không giám sát: tìm ra cấu trúc ẩn trong dữ liệu, còn RL là tối đa hóa điểm thưởng chứ không chỉ tìm cấu trúc
- Đánh đổi giữa Exploitation và Exploration
 - Exploitation: chọn những hành động mà sẽ dẫn đến kết quả tốt mà đã được học từ kinh nghiệm trước đó
 - Exploration: chọn các hành động mà trước chưa thử với kỳ vọng sẽ tìm được kết quả tốt hơn

⇒ Cố gắng cân bằng giữa Exploitation và Exploration: thử nhiều cách khác nhau rồi dần dần ưu tiên các hành động có vẻ mang lại kết quả tốt hơn
- Các thành phần của hệ thống Reinforcement Learning:

- Agent: là "bộ não" có khả năng học và đưa ra quyết định
- Environment: môi trường, không gian để Agent tương tác, và cũng tương tác ngược lại Agent thông qua việc ảnh hưởng trực tiếp lên các hành động và điểm thưởng
- Policy: quy tắc ứng xử của Agent khi ở một trạng thái và thời điểm nhất định, có thể hiểu là ánh xạ từ trạng thái (state) → hành động (action)
- Reward Signal: phần thưởng nhận được ngay lập tức sau khi agent đưa ra hành động
- Value Function: giá trị này ở một trạng thái nhất định là tổng phần thưởng kỳ vọng sẽ tích lũy được trong tương lai kể từ trạng thái đó. Agent đưa ra quyết định dựa trên cái này chứ không phải là dựa vào phần thưởng.
- Model of the environment: mô phỏng lại môi trường, dự đoán trạng thái và điểm thưởng của hành động dự định làm

2. Ví dụ Tic-Tac-Toe

- Thiết lập bài toán: ban đầu ta thiết lập trạng thái thắng có giá trị là 1, trạng thái thua/hòa có giá trị là 0, các trạng thái khác là 0.5
- Chọn hành động: tìm kiếm trạng thái hiện tại trong bảng trạng thái ta biết được giá trị của các bước đi tiếp theo
 - Tham lam: Phần lớn thời gian ta sẽ chọn nước đi có giá trị cao nhất
 - Khám phá: thỉnh thoảng ta sẽ chọn nước đi có giá trị không phải là lớn nhất để thử nghiệm các cách chơi khác mà trước đây chưa thử



- Cập nhật giá trị theo công thức $V(s) \leftarrow V(s) + \alpha[V(s') - V(s)]$. Giá trị của trạng thái ban đầu được kéo về gần hơn với giá trị của trạng thái mới. Ví dụ trạng thái cũ có giá trị là 0.5, giá trị mới có giá trị là 0.9 (khả năng thắng là rất cao) thì ta thấy nên tăng giá trị của trạng thái cũ lên bởi nó dẫn đến trạng thái có khả năng thắng rất cao. Giả sử $\alpha = 0.1$ thì giá trị của trạng thái cũ được cập nhật thành $0.5 + (0.9 - 0.5) * 0.1 = 0.54$. Nếu sau đó thật sự thắng ($V = 1$) thì ta sẽ quay lại tăng giá trị của chuỗi các trạng thái cũ dẫn đến kết quả thắng này.
- Sự vượt trội của RL với pp tiến hóa (Evolutionary): đó là thay vì chơi hết ván rồi mới đánh giá, và nếu ván đó thắng thì cho rằng mọi quyết định trong ván đó đều là tốt thì RL đánh giá theo từng bước đi, không cần chờ chơi hết ván và đánh giá riêng lẻ từng trạng thái, không phải cứ thắng thì có nghĩa là mọi trạng thái dẫn đến kết quả thắng đó đều là tốt.
- Lưu ý: khi mà số trạng thái trở nên lớn hơn thì ta sẽ không dùng bảng giá trị nữa mà dùng mạng nơ-ron để xử lý

2. Mô hình Markov

- Giả định markov: tương lai hoàn toàn độc lập với quá khứ nếu biết hiện tại
- Các thành phần của quá trình Markov:
 - Tập các trạng thái S
 - Mô hình chuyển trạng thái P: $P(s'|s)$ cho biết xác suất để từ trạng thái hiện tại s chuyển sang trạng thái s' là bao nhiêu
 - Ví dụ: $S = \{\text{nắng, mưa, nhiều mây}\}$, $P(\text{nắng}|\text{nắng}) = 0.6$, $P(\text{mưa}|\text{nắng}) = 0.3$, $P(\text{nhiều mây}|\text{nắng}) = 0.1$
- Chuỗi các trạng thái nối tiếp nhau tuân theo mô hình chuyển trạng thái là episodes (ví dụ: nắng → mây → mưa → mưa → nắng)
- Mỗi trạng thái sẽ có một điểm thưởng r và G_t (lợi tức) sẽ là tổng các phần thưởng nhận được từ thời điểm t đến khi kết thúc

$$G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots + \gamma^{H-1} r_{t+H-1}$$

- Với gamma thuộc khoảng $[0,1]$ là hệ số chiết khấu:
 - gamma = 0 chỉ quan tâm đến phần thưởng trước mắt
 - gamma = 1 phần thưởng ở tương lai xa cũng có mức độ ảnh hưởng như phần thưởng hiện tại
 - $0 < \text{gamma} < 1$: quan tâm cả phần thưởng ở cả hiện tại và tương lai nhưng quan tâm hơn ở tương lai gần, ở tương lai càng xa càng ít quan tâm

→ Nhưng không biết được ở tương lai t+1, t+2,... sẽ là trạng thái gì để biết r mà tính, tuy nhiên sự chuyển đổi trạng thái lại tuân theo mô hình chuyển trạng thái

→ Hàm giá trị $V(s)$: kỳ vọng tổng lợi tức tích lũy đến cuối nếu bắt đầu từ trạng thái s

$$V(s) = R(s) + \gamma \sum_{s' \in S} P(s'|s) V(s')$$

- Định nghĩa về MDP:

- Markov Decision Process is Markov Reward Process + actions
- Definition of MDP
 - S is a (finite) set of Markov states $s \in S$
 - A is a (finite) set of actions $a \in A$
 - P is dynamics/transition model for **each action**, that specifies $P(s_{t+1} = s' | s_t = s, a_t = a)$
 - R is a reward function¹

$$R(s_t = s, a_t = a) = \mathbb{E}[r_t | s_t = s, a_t = a]$$

- Discount factor $\gamma \in [0, 1]$
- MDP is a tuple: (S, A, P, R, γ)

¹Reward is sometimes defined as a function of the current state, or as a function of the (state, action, next state) tuple. Most frequently in this class, we will assume reward is a function of state and action

- Chính sách (Policy) quyết định thực hiện hành động gì ở mỗi trạng thái. Tổng quát hóa, ta có thể xem policy như một xác suất có điều kiện, với mỗi trạng thái s thì cung cấp 1 phân phối xác suất các hành động.
- Ở trạng thái s thực hiện theo chính sách π thì phần thưởng kỳ vọng sẽ nhận được là:

$$R^\pi(s) = \sum_{a \in A} \pi(a|s) R(s, a)$$

- Ở trạng thái s thực hiện theo chính sách π thì xác suất để trạng thái tiếp theo là trạng thái s' là:

$$P^\pi(s'|s) = \sum_{a \in A} \pi(a|s) P(s'|s, a)$$

- Thuật toán lặp (Iterative Algorithm): Cách tính hàm giá trị $V_\pi(s)$ cho một chính sách
 - bước 1: khởi tạo $V_0(s) = 0$ cho mọi trạng thái s
 - bước 2: Cập nhật giá trị V ở bước k dựa trên giá trị của bước trước đó $k-1$ cho tất cả các trạng thái, lặp đi lặp lại cho đến khi hội tụ (các con số không thay đổi nữa).
 - Công thức cập nhật (Bellman):

$$V_k^\pi(s) = \sum_a \pi(a|s) \left[R(s, a) + \gamma \sum_{s' \in S} p(s'|s, a) V_{k-1}^\pi(s') \right]$$

- Chính sách tối ưu: là chính sách giúp tác nhân đạt được hàm giá trị lớn nhất tại mọi trạng thái s :

$$\pi^*(s) = \arg \max_{\pi} V^\pi(s)$$

- Hàm giá trị tối ưu là duy nhất: khi xuất phát ở trạng thái s thì tổng phần thưởng tích lũy nhận tối đa tuyệt đối $V(s)$ là một hằng số với mọi s .
- Tác nhân không cần đưa ra hành động một cách ngẫu nhiên hay theo xác suất, khi ở trạng thái s nó sẽ thực hiện hành động a với xác suất 100%.
- Không phụ thuộc vào thời gian: ở mọi thời điểm, cứ rơi vào trạng thái s thì đưa ra đúng hành động a .

- Dù giá trị tối ưu là duy nhất nhưng có thể có nhiều chính sách có thể đạt được giá trị này.
- Thuật toán lặp chính sách (Policy iteration):
 - Chọn ngẫu nhiên một chính sách
 - Đánh giá chính sách được chọn bằng thuật toán lặp (dùng công thức bellman để cập nhật như đã nói ở trên)
 - Cải thiện chính sách bằng cách: ở mỗi trạng thái s ta thử xem là nếu làm một hành động khác đi so với thói quen cũ và sau đó lại tiếp tục thực hiện theo thói quen cũ thì có làm tăng tổng phần thưởng tích lũy không, hành động mang lại phần thưởng nhiều nhất thì sẽ được ghi đè → hình thành lên thói quen mới tốt hơn.

- Compute state-action value of a policy π_i
 - For s in S and a in A :

$$Q^{\pi_i}(s, a) = R(s, a) + \gamma \sum_{s' \in S} P(s'|s, a) V^{\pi_i}(s')$$

- Compute new policy π_{i+1} , for all $s \in S$

$$\pi_{i+1}(s) = \arg \max_a Q^{\pi_i}(s, a) \quad \forall s \in S$$

- Kiểm tra hội tụ: khi chính sách mới và chính sách cũ không khác gì nhau, không làm tăng hàm giá trị nữa thì lúc đó hội tụ
- Note: all the below is for finite state-action spaces
- Set $i = 0$
- Initialize $\pi_0(s)$ randomly for all states s
- While $i == 0$ or $\|\pi_i - \pi_{i-1}\|_1 > 0$ (L1-norm, measures if the policy changed for any state):
 - $V^{\pi_i} \leftarrow$ MDP V function policy **evaluation** of π_i
 - $\pi_{i+1} \leftarrow$ Policy **improvement**
 - $i = i + 1$
- Thuật toán lặp giá trị (Value Iteration)

- Bước 1: Khởi tạo hàm giá trị tại mọi trạng thái s đều bằng 0: $V_0(s) = 0$
- Bước 2 (Vòng lặp cập nhật): tại mỗi trạng thái, ta chọn hành động làm cho hàm giá trị lớn nhất; dừng vòng lặp khi hàm giá trị ở vòng lặp này so với vòng lặp trước nhỏ hơn một số epsilon
- Khi hàm giá trị V đã hội tụ thì ta sẽ lấy ra các hành động tương ứng để tạo thành chính sách tối ưu

- Set $k = 1$
- Initialize $V_0(s) = 0$ for all states s
- Loop until convergence: (for ex. $\|V_{k+1} - V_k\|_\infty \leq \epsilon$)
 - For each state s

$$V_{k+1}(s) = \max_a \left[R(s, a) + \gamma \sum_{s' \in S} P(s'|s, a) V_k(s') \right]$$

- Equivalently, in Bellman backup notation

$$V_{k+1} = BV_k$$

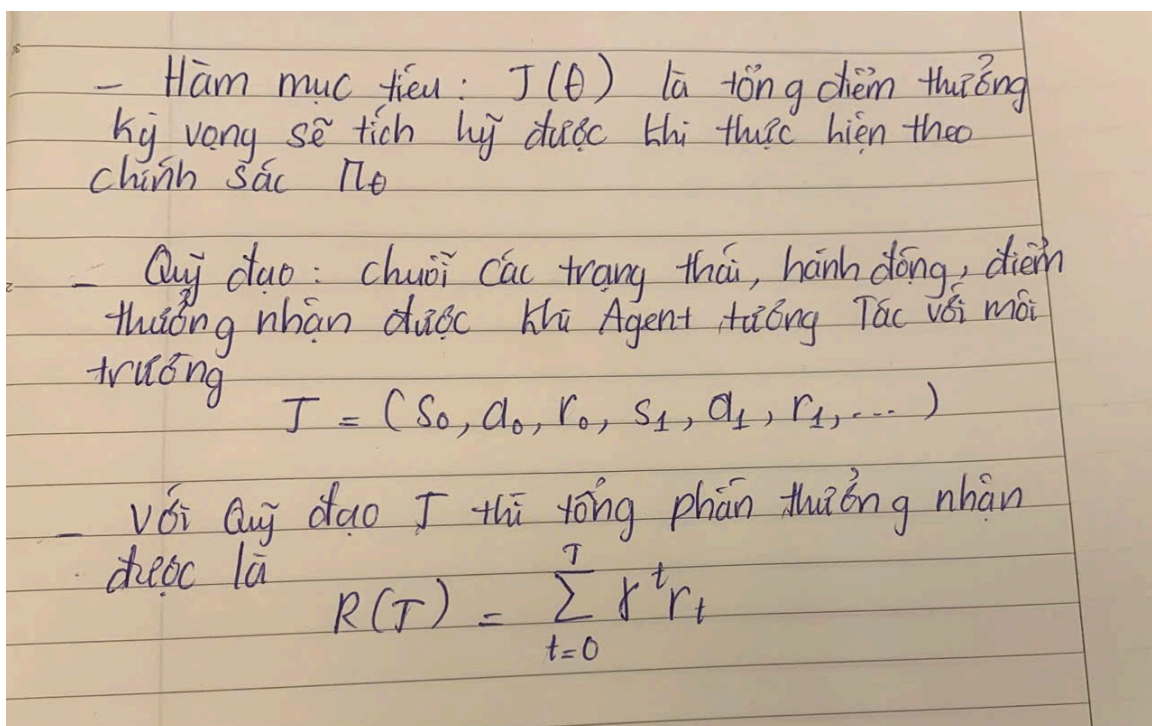
- To extract optimal policy if can act for $k + 1$ more steps,

$$\pi(s) = \arg \max_a \left[R(s, a) + \gamma \sum_{s' \in S} P(s'|s, a) V_{k+1}(s') \right]$$

- Giới hạn của các phương pháp và Giả định Markov:
 - Giả định Markov là tương lai độc lập với quá khứ nếu ta biết hiện tại → nhiều bài toán trong thực tế không thỏa mãn điều kiện này, ví dụ: khi khám cho bệnh nhân không chỉ nhìn vào triệu chứng ngay ở hiện tại mà còn phải xem tiền sử bệnh án
 - Với các phương pháp lập chính sách và lập giá trị ta buộc phải xây dựng mô hình xác suất hoàn hảo tuy nhiên việc tính chính xác xác suất để từ trạng thái s khi thực hiện hành động a sẽ chuyển sang trạng thái s' là rất khó
 - Khi không gian trạng thái lớn ta không thể duyệt qua hết tất cả các trạng thái để tính hàm Q hay bellman backup được.

3. Policy Gradient

- Những hạn chế của phương pháp Value-based (Q-learning) → xây dựng chính sách dựa vào việc chọn hành động a khi ở trạng thái s sao cho hành động a mang lại tổng điểm thưởng kỳ vọng nhận được là cao nhất:
 - Vấn đề với hành động liên tục và High-dimensional: giả sử góc quay từ 0 đến 360 độ, nó không thể tính toán được phần thưởng cho tất cả các góc quay (nếu quay góc 15 độ thì phần thưởng là bao nhiêu? 15.2 độ thì sao?); khi số lượng hành động quá lớn việc tính $\arg \max$ là cực kỳ tốn tài nguyên
 - Chiến thuật kém linh hoạt: ở mỗi trạng thái thường chọn cố định hành động tốt nhất đã được học, nhưng nếu chỉ tuân theo chiến thuật cố định này rất dễ bị "bắt bài" (điều này khá giống với phê phán Lucas trong tài chính khi mà cố áp đặt một quy luật tĩnh lên môi trường động)
 - Độ ổn định thấp: chỉ một thay đổi nhỏ ở hàm Q đã có thể ảnh hưởng lên việc lựa chọn hành động
- Định nghĩa:
 -



- Vì môi trường mang tính ngẫu nhiên, xác suất để chính sách chọn một hành động cụ thể cũng là ngẫu nhiên theo phân phối, nên một quỹ đạo cụ thể sẽ xuất hiện với xác suất:

1. Xác suất của trạng thái bắt đầu: $P(s_0)$
2. Quyết định của chính sách: $\pi_\theta(a_t|s_t)$
3. Phản hồi của môi trường (Transition Dynamics): $P(s_{t+1}|s_t, a_t)$

Công thức tổng quát của nó là:

$$P(\tau|\theta) = P(s_0) \prod_{t=0}^T \pi_\theta(a_t|s_t) P(s_{t+1}|s_t, a_t)$$

- Tổng số điểm kỳ vọng sẽ nhận được nếu theo chính sách pi (bộ tham số là phi - tham số của mạng nơ ron):

$$J(\theta) = \sum_{\tau} P(\tau|\theta) R(\tau)$$

- Mục tiêu của ta là cần phải tìm ra bộ tham số để hàm J đạt giá trị lớn nhất:

$$\theta^* = \arg \max_{\theta} J(\theta)$$

- Để tìm được bộ tham số tốt nhất ta phải tính được đạo hàm của hàm J từ đó cập nhật các tham số phi:

$$\nabla_{\theta} J(\theta) = \nabla_{\theta} \sum_{\tau} P(\tau|\theta) R(\tau)$$

$$\nabla_{\theta} J(\theta) = \sum_{\tau} \nabla_{\theta} P(\tau|\theta) R(\tau)$$

$$\frac{d}{dx} \ln(f(x)) = \frac{f'(x)}{f(x)}$$

Áp dụng y hệt công thức này cho gradient của xác suất $P(\tau|\theta)$ theo θ , ta có:

$$\nabla_{\theta} \log P(\tau|\theta) = \frac{\nabla_{\theta} P(\tau|\theta)}{P(\tau|\theta)}$$

Bây giờ, chỉ cần nhân chéo $P(\tau|\theta)$ lên, ta được:

$$\nabla_{\theta} \mathbf{P}(\tau|\theta) = \mathbf{P}(\tau|\theta) \nabla_{\theta} \log \mathbf{P}(\tau|\theta)$$

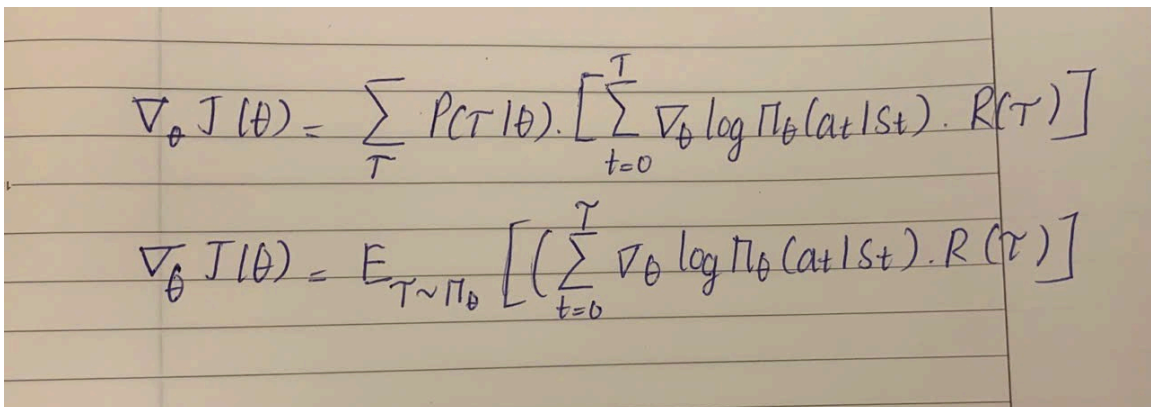
ĐÂY GỌI LÀ LOG-DERIVATIVE TRICK

$$\log P(\tau|\theta) = \log P(s_0) + \sum_{t=0}^T \log \pi_{\theta}(a_t|s_t) + \sum_{t=0}^T \log P(s_{t+1}|s_t, a_t)$$

Do $P(s_0)$ và $P(s_{t+1} | s_t, a_t)$ đều không phụ thuộc vào θ nên đạo hàm của nó sẽ bằng 0

$$\nabla_{\theta} \log P(\tau|\theta) = \sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t|s_t)$$

Từ các công thức trên ta được:



$$\nabla_{\theta} J(\theta) = \sum_{\tau} P(\tau|\theta) \cdot \left[\sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t|s_t) \cdot R(\tau) \right]$$

$$\nabla_{\theta} J(\theta) = E_{\tau \sim \pi_{\theta}} \left[\left(\sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t|s_t) \right) \cdot R(\tau) \right]$$

Kết hợp với việc ta không thể tính toán hết các quỹ đạo có thể xảy ra nên ta sẽ chơi N ván bất kỳ rồi lấy trung bình và việc quyết định thực hiện hành

động nào ở thời điểm t sẽ ảnh hưởng đến các hành động, trạng thái ở sau đó theo một hệ số gamma chứ không ảnh hưởng đến các hành động, trạng thái đã xảy ra trong quá khứ ta có công thức cuối cùng sẽ là:

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \left[\sum_{t=0}^{T_i} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \cdot G_t \right]$$

□ ∴

$$G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots$$

4. Actor critic

- Thay vì phải chờ đến cuối ván mới có thể cập nhật trọng số, ta chọn cách thêm 1 đầu ra để dự đoán giá trị điểm thưởng tích lũy kỳ vọng sẽ nhận được $V(s_t)$ từ đó tính ra delta để đánh giá hành động được thực hiện ở thời điểm hiện tại là tốt hay không với delta được tính bằng công thức:

$$\delta_t = r_t + \gamma V_{\phi}(s_{t+1}) - V_{\phi}(s_t)$$

- Sau khi thực hiện hành động thì nhận được điểm thưởng r_t và chuyển sang trạng thái s_{t+1} , delta có thể hiểu là độ sai lệch giữa dự đoán và "thực tế" sau 1 bước. Nếu $\delta_t > 0$ có nghĩa là hành động vừa rồi dẫn đến kết quả tốt hơn dự đoán ban đầu nên ta cần tăng xác suất hành động đó khi ở trạng thái s lên, với $\delta_t < 0$ thì ngược lại
- Khi đã có delta ta cập nhật trọng số bằng loss của critic (bình phương delta) và loss actor

$$-\log \pi_{\theta}(a_t | s_t) \times \delta_t.$$

- Cứ cập nhật như thế ở mỗi bước đến hết ván chơi.
- Cụ thể cách code và ưu nhược điểm của phương pháp này được giải thích và nói rõ trong file [CartPole.ipynb](#)